An Intro to Natural Language Processing Part II

GENERAL ASSEMBLY

Agenda

- Structured vs Unstructured Data
- Vectorizing our data
- The Count Vectorizer
- The TF-IDF Vectorizer
- **Appendix:** The Naive Bayes Model

Structured vs Unstructured Data

In data science, we'll often refer to **structured data** or **unstructured data**.

- **structured data** is data that neatly fit into rows and columns, where one row = one observation and one column = one variable.
- **unstructured data** is everything else

 © 2024 General Assembly

Unstructured Data

- Plain text
- Images
- Videos
- Audio files
- JSON
- HTML

 © 2024 General Assembly

Unstructured Data

- Plain text
- Images
- Videos
- Audio files

“Semistructured” Data

- JSON
- HTML

Some people make an alternate definition for data that aren’t tabular but still have some internal structure to them

Structured vs Unstructured Data

Data scientists need to be able to work with all data, but ML models can ***only*** work with structured data.

One of the most difficult tasks in data science is to correctly process unstructured data so that it becomes usable in an ML model. Turning unstructured data into tabular data could be called **feature extraction**.

For text data, that's our job today!

 © 2024 General Assembly

Vectorizing Text Data

But first...

Step 0: **Normalize your corpus!** You must preprocess your data before any feature extraction is to begin. We want “Cat”, “cat”, and “cats” to be the same word when it comes to predictive modeling.

Count Vectorization

Count vectorizing text data is simple: For each term in the corpus's vocabulary, we create a column. The values in that column correspond to the **count** of occurrences in that document. For example:

Count Vectorization

Document	Normalized Document
the cat sat on the mat	the cat sit on the mat
the dog played fetch	the dog play fetch
cats and dogs playing together	cat and dog play together

Count vectorized table ready for modeling!

the	cat	sit	on	mat	dog	play	fetch	and	together
2	1	1	1	1	0	0	0	0	0
1	0	0	0	0	1	1	1	0	0
0	1	0	0	0	1	1	0	1	1

EDA after Count Vectorization

In addition to having a model-ready dataframe, we can use this for EDA! Column sums can get us word frequencies that we can make into cool viz.

the	cat	sit	on	mat	dog	play	fetch	and	together
2	1	1	1	1	0	0	0	0	0
1	0	0	0	0	1	1	1	0	0
0	1	0	0	0	1	1	0	1	1

 © 2024 General Assembly

Bag-of-Words

You might notice that something is lost here! Count vectorizing no longer preserves the order of terms in your document.

You'd be surprised how much word order doesn't matter. Problems that do this are so-called **bag-of-words** models.

Let’s try this out!



The TF-IDF Vectorizer

Count Vectorizer Drawback

The count vectorizer is intuitive - but is it optimal? When using these counts in a model, we're making the implicit assumption that, for a term in our vocabulary:

more occurrences = more important

is this necessarily true?

 © 2024 General Assembly

The TF-IDF Vectorizer

There's an alternative vectorizing technique: **the TF-IDF vectorizer**. As the name suggests, it's made up of two quantities:

- **T**erm **F**requency = The proportion of times the term appears in the given document
- (**I**nverse) **D**ocument **F**requency = (One divided by) the proportion of documents the given term appears in

 © 2024 General Assembly

The TF-IDF Vectorizer

Generally, you can think of thee TF-DF score for a given term in a document as:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t) = \frac{\text{tf}(t,d)}{\text{df}(t)}$$

However, formulas used in practice offer some adjustments to prevent zero-division and bias towards longer documents. **Calculation of TF-IDF can even differ across different programming languages and libraries! Read the documentation!**

 © 2024 General Assembly

The TF-IDF Vectorizer

The logic behind this is we want to determine which terms in our corpus are the most **informative**.

A term is informative if it appears at least once in a document (high **tf**) but in not too many documents (low **df**), thus resulting in a high **TF-IDF**.

(Aside: Recall this vectorization process is unsupervised - we aren't using our *y*-variable at all here! Which terms *actually* prediction *y* is unknown to us at this time.)

 © 2024 General Assembly

TF-IDF Example (using simplest formula)

Recall our corpus from earlier:

the	cat	sit	on	mat	dog	play	fetch	and	together
2	1	1	1	1	0	0	0	0	0
1	0	0	0	0	1	1	1	0	0
0	1	0	0	0	1	1	0	1	1

 © 2024 General Assembly

TF-IDF Example (using simplest formula)

TF matrix

the	cat	sit	on	mat	dog	play	fetch	and	together
1/3	1/6	1/6	1/6	1/6	0	0	0	0	0
1/4	0	0	0	0	1/4	1/4	1/4	0	0
0	1/5	0	0	0	1/5	1/5	0	1/5	1/5

×

IDF matrix

the	cat	sit	on	mat	dog	play	fetch	and	together
log(3/2)	log(3/2)	log(3)	log(3)	log(3)	log(3/2)	log(3/2)	log(3)	log(3)	log(3)

=

TF-IDF matrix

the	cat	sit	on	mat	dog	play	fetch	and	together
0.135	0.068	0.183	0.183	0.183	0	0	0	0	0
0.101	0	0	0	0	0.101	0.101	0.274	0	0
0	0.081	0	0	0	0.081	0.081	0	0.220	0.220

Python Time!



 © 2024 General Assembly

Appendix: Naive Bayes

Bayes' Theorem

Remember the **"And" Rule** for probability:

$$P(A \cap B) = P(A)P(B|A)$$

This can be rearranged to form **Bayes' Theorem** for conditional probability:

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

 © 2024 General Assembly

Bayes' Theorem

Since we're doing classification, let *y* be an observation's true class. We'll let ***x*** represent all of the *x*-variables in our model:

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$

Our prediction will be the value of *y* that is the most likely outcome

 © 2024 General Assembly

Bayes' Theorem

Note the denominator doesn't matter, since it's constant with respect to *y*.

$$P(y|\mathbf{x}) \propto P(\mathbf{x}|y)P(y)$$

 © 2024 General Assembly

The Naive Part

The ***naive*** part of Naive Bayes is that we assume each *x*-variable is independent. This is an incorrect assumption since for us, each *x*-variable is the occurence of a word in a document. Since documents are plain English speech, the words are definitely not independent! In practice, we get good results, so this model is useful.

$$
\begin{aligned}
P(y|\mathbf{x}) &\propto P(\mathbf{x}|y)P(y) \\
&= P(x_1|y)P(x_2|y)\cdots P(x_N|y)P(y) \\
&= P(y)\prod P(x_i|y)
\end{aligned}
$$

 © 2024 General Assembly